

DOCUMENTATION

LONG SHORT-TERM MEMORY(LSTM)

Long Short-Term Memory (LSTM) is a special type of Recurrent Neural Network (RNN) designed to learn and remember long-term dependencies in sequential data. Unlike traditional RNNs, which struggle to retain information over long sequences due to the vanishing gradient problem, LSTM uses a memory cell and three gates—forget gate, input gate, and output gate—to control the flow of information. The forget gate decides which past information should be removed, the input gate determines what new information should be stored, and the output gate controls what information is passed to the next time step. By carefully managing memory through these gates using sigmoid and tanh activation functions, LSTM can effectively remember important information for long periods. Because of this ability, LSTM is widely used in applications such as speech recognition, machine translation, sentiment analysis, text generation, and time-series forecasting.

Importing Required Libraries

```
import numpy
import matplotlib.pyplot as plt
import pandas
import math
from keras.models import Sequential
from keras.layers import Dense
from keras.layers import LSTM
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error
```

This code imports all the required libraries for building and evaluating an LSTM-based deep learning model, commonly used for time-series prediction. `numpy` and `math` are used for numerical computations, while `pandas` helps in loading and handling structured data. `matplotlib.pyplot` is used for visualizing data and model results through graphs. From Keras, `Sequential` is imported to create a sequential neural network model, `Dense` is used for fully connected output layers, and `LSTM` is used to add Long Short-Term Memory layers that can learn patterns from sequential data. `MinMaxScaler` from `sklearn.preprocessing` is used to normalize the data so that values fall within a specific range (usually 0 to 1), which improves model performance. Finally, `mean_squared_error` from `sklearn.metrics` is used to measure the difference between predicted values and actual values, helping evaluate how well the model performs.

Data Loading and Normalization

```
dataframe=pandas.read_csv(r"C:\Users\asus\Downloads\Deeplearningdataset\airline-passengers.csv",usecols=[1],engine='python')
dataset=dataframe.values
dataset=dataset.astype("float32")
scaler=MinMaxScaler(feature_range=(0,1))
dataset=scaler.fit_transform(dataset)
```

This code loads and prepares data for an LSTM model. First, the CSV file containing airline passenger data is read using `pandas.read_csv`, where only the second column (`usecols=[1]`) is selected because it contains the passenger values, and the Python engine is used for compatibility. The loaded data is then converted into a NumPy array using `.values` so it can be processed by machine learning models. After that, the data

type is changed to float32, which is required for efficient computation in deep learning frameworks like Keras. Next, a MinMaxScaler is created to scale all values between 0 and 1, ensuring that the LSTM model trains faster and more accurately. Finally, fit_transform is applied to normalize the entire dataset, transforming the original values into a scaled range suitable for training the neural network.

Splitting the Dataset into Training and Testing Sets

```
train_size=int(len(dataset)*0.67)
test_size=len(dataset)-train_size
train,test=dataset[0:train_size,:],dataset[train_size:len(dataset),:]
print(len(train),len(test))
```

96 48

This code divides the dataset into training and testing parts for model development and evaluation. First, train_size is calculated as 67% of the total dataset length using len(dataset) * 0.67, and it is converted to an integer to avoid decimal values. The remaining data is assigned to test_size by subtracting the training size from the total dataset length. Next, the dataset is sliced using array indexing: the first 67% of the data is stored in train, and the remaining 33% is stored in test. Finally, print(len(train), len(test)) displays the number of samples in the training and testing datasets, confirming the split (for example, 96 training samples and 48 testing samples).

Creating Input–Output Sequences for LSTM

```
def create_dataset(dataset,look_back=1):
    dataX,dataY=[],[]
    for i in range(len(dataset)-look_back-1):
        a=dataset[i:(i+look_back),0]
        dataX.append(a)
        dataY.append(dataset[i+look_back,0])
    return numpy.array(dataX),numpy.array(dataY)
```

This function prepares the dataset in a format suitable for training an LSTM model by converting time-series data into input-output pairs. The function `create_dataset` takes the dataset and a parameter called `look_back`, which defines how many previous time steps are used to predict the next value. Inside the function, two empty lists, `dataX` and `dataY`, are created to store input sequences and corresponding target values. A loop runs through the dataset, and for each position, a sequence of values of length `look_back` is extracted and added to `dataX`, while the value immediately following that sequence is added to `dataY`. Finally, both lists are converted into NumPy arrays and returned, making the data ready to be fed into an LSTM model for learning temporal patterns.

Creating Sequences and Reshaping Data for LSTM Input

```
look_back=1
trainX,trainY=create_dataset(train,look_back)
testX,testY=create_dataset(test,look_back)
trainX=numpy.reshape(trainX,(trainX.shape[0],1,trainX.shape[1]))
testX=numpy.reshape(testX,(testX.shape[0],1,testX.shape[1]))
```

```
print(trainX.shape)
```

```
(94, 1, 1)
```

This code prepares the training and testing data in the correct format required by an LSTM model. First, `look_back` is set to 1, meaning the model will use one previous time step to predict the next value. The `create_dataset` function is then applied to both the training and testing datasets to generate input sequences (`trainX`, `testX`) and their corresponding target values (`trainY`, `testY`). After creating these sequences, the input data is reshaped using `numpy.reshape` into a three-dimensional format (`samples`, `time_steps`, `features`), which is mandatory for LSTM models. Here, each sample contains 1 time step and 1 feature. Finally, `print(trainX.shape)` displays the shape

of the training input data, and the output (94, 1, 1) confirms that there are 94 samples, each with 1 time step and 1 feature, making the data ready for LSTM training.

Building and Training the LSTM Model

```
model=Sequential()  
model.add(LSTM(4,input_shape=(1,look_back)))  
model.add(Dense(1))  
model.compile(loss="mean_squared_error",optimizer="adam")  
model.fit(trainX,trainY,epochs=100,batch_size=1,verbose=2)
```

This code creates and trains an LSTM-based neural network using the Keras Sequential API. First, a Sequential model is initialized to stack layers one after another. An LSTM layer with 4 memory units is then added, where `input_shape=(1, look_back)` specifies that the model will receive one time step with a number of features equal to `look_back`. Next, a Dense layer with one neuron is added to produce a single output value, which is suitable for regression problems like time-series prediction. The model is then compiled using mean squared error as the loss function to measure prediction error and the Adam optimizer to efficiently update model weights. Finally, the model is trained using the training data for 100 epochs with a batch size of 1, and `verbose=2` is used to display training progress in a readable format.

Checking the Shape of Output (Target) Data

```
print(trainY.shape)  
print(testY.shape)
```

```
(94,)  
(46,)
```

This code prints the shapes of the target output arrays used for training and testing the LSTM model. `print(trainY.shape)` displays the number of output values corresponding to the training input sequences, while `print(testY.shape)` shows the number of output values for the testing data. The output (94,) means there are 94 target values for training, and (46,) means there are 46 target values for testing. These values match the number of input samples created earlier, confirming that for each

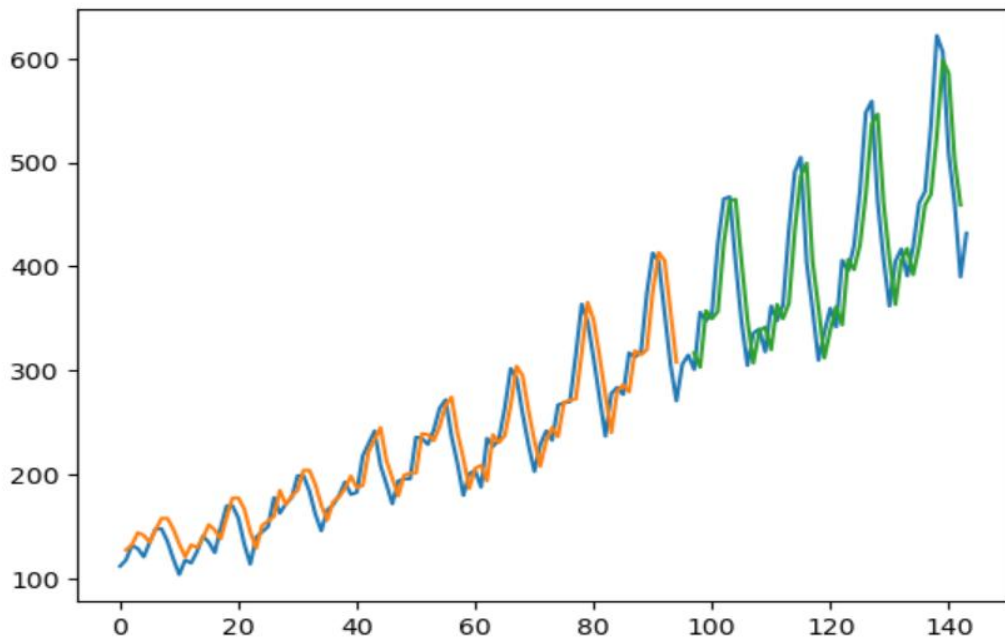
input sequence the model has exactly one corresponding output value, which is required for proper training and evaluation of the model.

Making Predictions, Evaluating Performance, and Visualizing Results

```
trainpredict=model.predict(trainX)
testpredict=model.predict(testX)
trainpredict=scaler.inverse_transform(trainpredict)
trainY=scaler.inverse_transform([trainY])
testpredict=scaler.inverse_transform(testpredict)
testY=scaler.inverse_transform([testY])
trainScore=math.sqrt(mean_squared_error(trainY[0],trainpredict[:,0]))
testScore=math.sqrt(mean_squared_error(testY[0],testpredict[:,0]))
trainpredictplot=numpy.empty_like(dataset)
trainpredictplot[:,:]=numpy.nan
trainpredictplot[look_back:len(trainpredict)+look_back,:]=trainpredict
testpredictplot=numpy.empty_like(dataset)
testpredictplot[:,:]=numpy.nan
testpredictplot[len(trainpredict)+(look_back*2)+1:len(dataset)-1,:]=testpredict
plt.plot(scaler.inverse_transform(dataset))
plt.plot(trainpredictplot)
plt.plot(testpredictplot)
plt.show()
```

3/3 ————— 1s 268ms/step

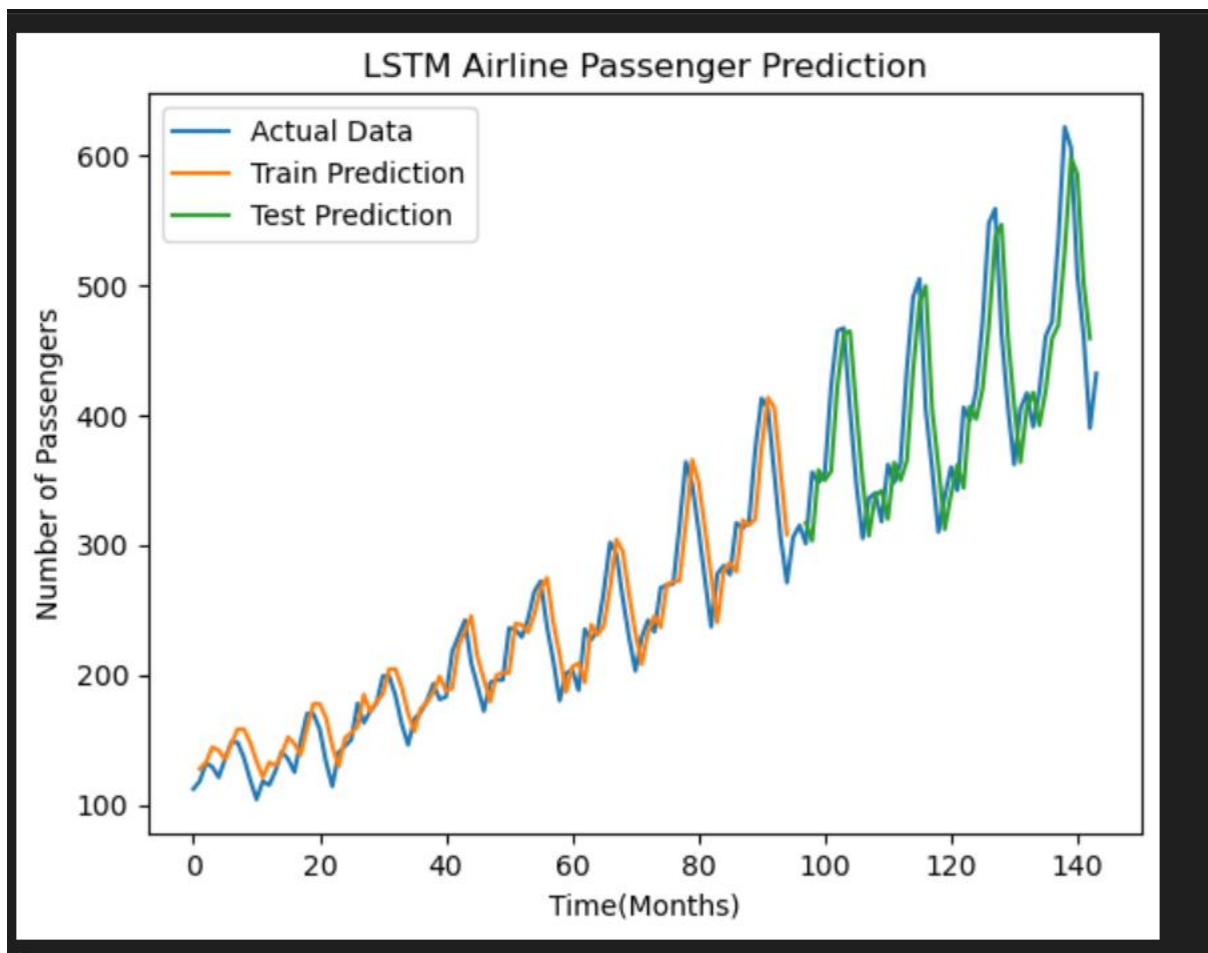
2/2 ————— 0s 62ms/step



This code uses the trained LSTM model to generate predictions and then evaluates and visualizes the results. First, predictions are made on both training data (trainX) and testing data (testX) using `model.predict`. Since the model was trained on scaled data, the predicted values and the actual target values are converted back to their original scale using `scaler.inverse_transform` for meaningful interpretation. The model's performance is then measured using Root Mean Squared Error (RMSE), which is calculated by applying the square root to the mean squared error between the actual and predicted values for both training and testing datasets. To visualize the predictions, empty arrays filled with NaN values are created so the predicted points align correctly with the original time-series data. Training predictions are placed at the appropriate positions, followed by test predictions, ensuring continuity in the plot. Finally, the original dataset, training predictions, and testing predictions are plotted together, allowing a clear visual comparison between actual values and the model's predictions.

Visualizing Actual and Predicted Airline Passenger Data

```
plt.plot(scaler.inverse_transform(dataset),label="Actual Data")
plt.plot(trainpredictplot,label="Train Prediction")
plt.plot(testpredictplot,label="Test Prediction")
plt.xlabel("Time(Months)")
plt.ylabel("Number of Passengers")
plt.title("LSTM Airline Passenger Prediction")
plt.legend()
plt.show()
```

This code creates a clear and labeled plot to compare the actual airline passenger data with the LSTM model's training and testing predictions. First, the original dataset is converted back to its real scale using `inverse_transform` and plotted as the actual data. Then, the training predictions and testing predictions are plotted on the same graph, each with a descriptive label to distinguish them. The x-axis is labeled as time in months, and the y-axis represents the number of passengers, making the graph easy to interpret. A meaningful title is added to describe the purpose of the plot, and `plt.legend()` displays a legend that identifies each line. Finally, `plt.show()` displays the complete graph, allowing a visual comparison of how closely the LSTM predictions follow the actual data.